Reward Design with Generalized Prioritized Sweeping in Taxi-v4

Student: Alish

Course: Reinforcement Learning

Date: July 2026

1. AI Use Disclaimer

I used AI assistance, mainly ChatGPT/Codex, during this project. The assistance was used for debugging, checking the project requirements, improving the experiment runner, structuring the report, and clarifying reinforcement learning concepts. AI also helped draft and revise some report text. The implementation choices, experiment interpretation, and final responsibility for the submission remain mine. The main algorithmic components, reward designs, and conclusions were reviewed against the project instructions and the actual code/results in this repository.

2. Work Beyond the Minimum Requirements

The minimum project requires four reward designs, SARSA, prioritized sweeping, a generalization mechanism, reward comparison, parameter sweeps, component ablations, plots, and policy visualization. This project includes those core parts and adds several extra pieces:

- Five reward designs are implemented instead of four.
- The fifth reward is a custom Composite Remaining Path reward.
- The reward comparison plot uses a normalized environment reward so different training reward scales do not distort the main comparison.
- Trained Q-tables are saved under `results/models`, so policies can be visualized without retraining.
- `explain_simulation.py` provides a step-by-step explanation of one learned episode.
- A `--fast` mode gives a short single-seed development run.

The extra reward design is discussed in Section 5.5. The normalized reward comparison is discussed in Section 9.2. The saved-policy visualization workflow is discussed in Section 8 and Section 13.

3. Environment and Map

The project uses Gymnasium Taxi-v4 with the default Taxi map. The code attempts to create the environment with `is_rainy=False`, as required. If the installed Gymnasium version does not support that argument, the code falls back to the standard deterministic Taxi-v4 environment.

Taxi-v4 has 500 discrete states and 6 actions: south, north, east, west, pickup, and dropoff. The state is decoded as:

```
(taxi_row, taxi_col, passenger_location, destination)
```

The passenger can be at one of four landmarks or inside the taxi. The destination is one of the same four landmarks. This representation is used by the custom rewards and by the kernel generalization mechanism.

4. Algorithm Specification

The base learner is tabular SARSA with epsilon-greedy action selection. The SARSA update is:

```
Q(s, a) <- Q(s, a) + alpha * delta
delta = r + gamma * Q(s_next, a_next) - Q(s, a)
```

The implementation uses one agent class, `GeneralizedSweepingSARSAAgent`, with flags for prioritized sweeping and kernel generalization. This keeps the ablations consistent because the same code path is used with components switched on or off.

Main hyperparameters in the full framework:

```
alpha = 0.5
gamma = 0.99
epsilon starts at 0.3
epsilon_min = 0.01
planning_steps n = 5
theta = 1e-4
beta = 4.0
c = 0.5
max_kernel_neighbors = 4
kernel_scale = 0.5
```

The discount factor was fixed at gamma = 0.99 across all reward comparisons, parameter sweeps, and component ablations. I kept gamma constant intentionally because changing gamma would change the discounted MDP itself and would make the reward and algorithm comparisons less meaningful.

Epsilon decays over training. The decay is selected so exploration becomes much smaller after a substantial part of training, while still leaving a small amount of random action selection.

4.1 Prioritized Sweeping

After each real SARSA update, the agent stores the observed transition in an internal model:

```
M(s, a) = (r, s_next, done)
```

The absolute TD error is used as the priority. High-priority state-action pairs are pushed into a priority queue. During planning, the agent pops high-priority transitions and performs SARSA-style model updates. The next action during planning is selected with the same epsilon-greedy action rule, which keeps the update on-policy in the SARSA sense.

This is not simple Dyna-Q. Simple Dyna-Q would sample previously observed transitions more uniformly. Here, transitions with larger estimated learning impact receive planning attention first.

4.2 Kernel Generalization

The generalization mechanism updates nearby, semantically compatible state-action pairs after a real transition. The kernel is:

```
K(s, s_hat) = 1 / (1 + exp(beta * (d(s, s_hat) - c)))
```

Here, d is the Manhattan distance between taxi positions. The kernel is symmetric because the Manhattan distance is symmetric. Two Taxi states are considered candidates for generalization only when they have the same passenger status/location and the same destination. This is important because two nearby taxi positions can belong to completely different subtasks if the passenger or destination differs.

The kernel update uses the neighbor's own stored transition:

```
delta_hat = r_hat + gamma * Q(s_hat_next, a_hat_next) - Q(s_hat, a)
Q(s_hat, a) <- Q(s_hat, a) + alpha * K(s, s_hat) * delta_hat
```

The implementation applies kernel generalization only to movement actions. Pickup and dropoff are not generalized because their validity depends on exact location. This avoids spreading illegal pickup/dropoff values into nearby but invalid states.

5. Reward Designs

The project implements five rewards. The first two are required baselines, rewards 3 and 4 are custom required rewards, and reward 5 is an extra custom reward.

5.1 Base Environment Reward

The base reward is the reward returned by Taxi-v4:

```
R = R_base
```

This reward preserves the original environment objective. It is the most direct way to measure whether the agent is solving the task according to Taxi's native scoring.

5.2 Sparse Reward

Sparse reward ignores most feedback:

```
R = 1.0 if successful dropoff
R = 0.0 otherwise
```

This design is conceptually clean but difficult for learning. Most transitions provide no useful signal, so the agent must discover the full pickup-and-dropoff sequence before it receives meaningful feedback.

5.3 Reward 3: Dense Target Progress Reward

Reward 3 gives immediate feedback for moving toward the current subgoal. Before pickup, the target is the passenger. After pickup, the target is the destination.

```
progress = old_distance_to_target - new_distance_to_target

R = 0.75 * progress - 0.05
+ 2.0 for successful pickup
+ 5.0 for successful dropoff
- 2.0 for illegal pickup/dropoff
- 0.5 for wall bump
```

The intuition is that navigation should be reinforced before the final dropoff. This can reduce the exploration burden compared with sparse reward because the agent receives a signal even before completing the full task.

5.4 Reward 4: Subgoal Safety Reward

Reward 4 emphasizes safe task completion:

```
R = -0.10
+ 3.0 for successful pickup
+ 10.0 for successful dropoff
- 4.0 for illegal pickup/dropoff
- 1.0 for wall bump
- 0.25 when moving away from the current target
```

This reward is designed to strongly discourage illegal pickup/dropoff actions and wall bumps while still rewarding the two meaningful subgoals: pickup and final dropoff. It should help when illegal actions are the main failure mode, but the stronger penalties can also make learning more sensitive to alpha and planning depth.

5.5 Reward 5: Composite Remaining Path Reward

Reward 5 measures whether the estimated full remaining task becomes shorter. If the passenger is not inside the taxi:

```
remaining = distance(taxi, passenger) + distance(passenger, destination)
```

If the passenger is already inside the taxi:

```
remaining = distance(taxi, destination)
```

The reward is:

```
R = (remaining_before - remaining_after) / 16.0
```

```
- 0.02
+ 1.0 for successful dropoff
- 0.3 for illegal pickup/dropoff
```

This reward is a full replacement reward, not base reward plus shaping. It gives a task-level signal: the agent is rewarded when the whole remaining route is estimated to become shorter. A limitation is that Manhattan distance ignores Taxi walls, so the distance is a heuristic rather than an exact shortest path.

6. Summary of Findings

The main finding is that reward design, planning, and generalization interact strongly. The best reward is not simply the one with the largest internal training reward scale. For comparison across reward designs, the fairest metrics are environment reward, normalized environment reward, success rate, and illegal-action count.

In the latest reward comparison, Base Environment Reward and Composite Remaining Path Reward reached strong final performance. Over the last 50 episodes per seed, both reached 100% success in the saved results, with near-perfect normalized environment reward. Sparse Reward also improved, but it retained many illegal actions and lower environment reward because the agent receives little guidance before success. Reward 3 and Reward 4 were more difficult under the shared full-framework hyperparameters. This does not mean the reward ideas are useless; it means their scales and penalties interact with alpha, planning, and exploration.

The ablation results show that all base-reward setups eventually reached high success, but their final environment reward and illegal-action behavior differed. Kernel generalization alone was particularly strong in the final tail. Prioritized sweeping is useful, but with imperfect early values it can also replay and amplify early estimates. The full framework remains important because it combines real SARSA updates, model-based replay, and local value propagation, but it requires careful hyperparameter selection.

The parameter sweep supports this interpretation. For the base reward, alpha = 0.8 and n = 10 produced the best final environment reward among the tested sweep settings. For Reward 4, alpha = 0.8 was much better than alpha = 0.2, while increasing planning from n = 2 to n = 10 did not improve the final score. This suggests that the designed safety reward needs stronger learning updates, but too much planning can amplify noisy penalty-heavy estimates.

7. Experiment Setup

The latest available result files use:

```
Environment: Taxi-v4, is_rainy=False when supported
Seeds: [1, 2, 3, 4, 5]
Reward comparison episodes: 500
Component ablation episodes: 500
Parameter sweep episodes: 400
Smoothing window: 50 episodes
Confidence interval: 95% across 5 seeds
```

The reward comparison trains the full framework on all five rewards. The component ablation compares four setups under the base reward:

- Pure SARSA.
- SARSA + Prioritized Sweeping.
- SARSA + Kernel.
- Full Framework: SARSA + Prioritized Sweeping + Kernel.

The parameter sweep evaluates two parameters:

```
alpha in {0.2, 0.8}
planning_steps n in {2, 10}
```

The sweep is run for Base Environment Reward and Reward 4, so both the base reward and a custom designed reward are evaluated in the sweep section.

The main metrics are:

- Environment reward: Taxi-v4's original reward accumulated over an episode.
- Normalized environment reward: environment reward scaled relative to the optimal reward for the episode's start state.
- Success rate: whether the episode ended with successful dropoff.
- Illegal actions: number of illegal pickup/dropoff actions.
- Episode length: number of steps taken.

I did not use training reward as a cross-reward comparison metric because the designed rewards have different scales. I also avoided claiming unique-state and mean-TD-error plots in the final report because those metrics are not currently saved in the main result CSV files. The analysis therefore focuses on metrics that are present, reproducible, and comparable across the saved runs.

8. Best Policy Visualization

The project supports Gymnasium's Taxi visualization. Saved policies can be animated without retraining:

```
python main.py --visualize-saved
```

In addition to the live command, I generated one successful saved-policy animation for each reward design. The GIF files are included in `reports/policy_animations/`:

```
base_policy.gif
sparse_policy.gif
reward_3_policy.gif
reward_4_policy.gif
reward_5_policy.gif
```

The GIFs were produced from Gymnasium `render_mode="rgb_array"` frames using saved Q-tables from `results/models`. The following contact sheet shows the final frame from each successful animation:

Figure: Best policy animation contact sheet

The same policies can also be visualized live with:

```
python main.py --visualize-saved --visualize-reward base --seed 1
python main.py --visualize-saved --visualize-reward sparse --seed 1
python main.py --visualize-saved --visualize-reward reward_3 --seed 1
python main.py --visualize-saved --visualize-reward reward_4 --seed 1
python main.py --visualize-saved --visualize-reward reward_5 --seed 1
```

During development, some saved policies succeeded immediately while others entered loops for certain seeds. This is a useful reminder that a single animation is not the whole evaluation. The plots and confidence intervals are needed because Taxi learning is seed-sensitive.

9. Results and Deep Interpretation

9.1 Component Ablation

The ablation compares the contribution of prioritized sweeping and kernel generalization under the base reward.

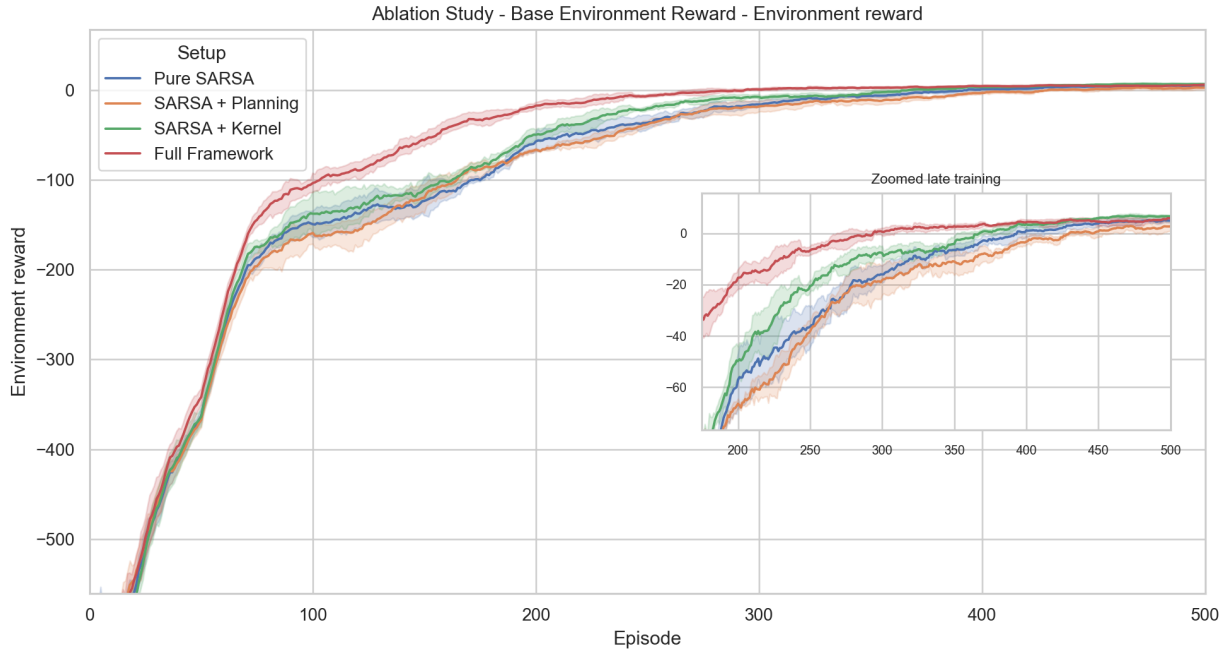

Figure: Ablation base environment reward

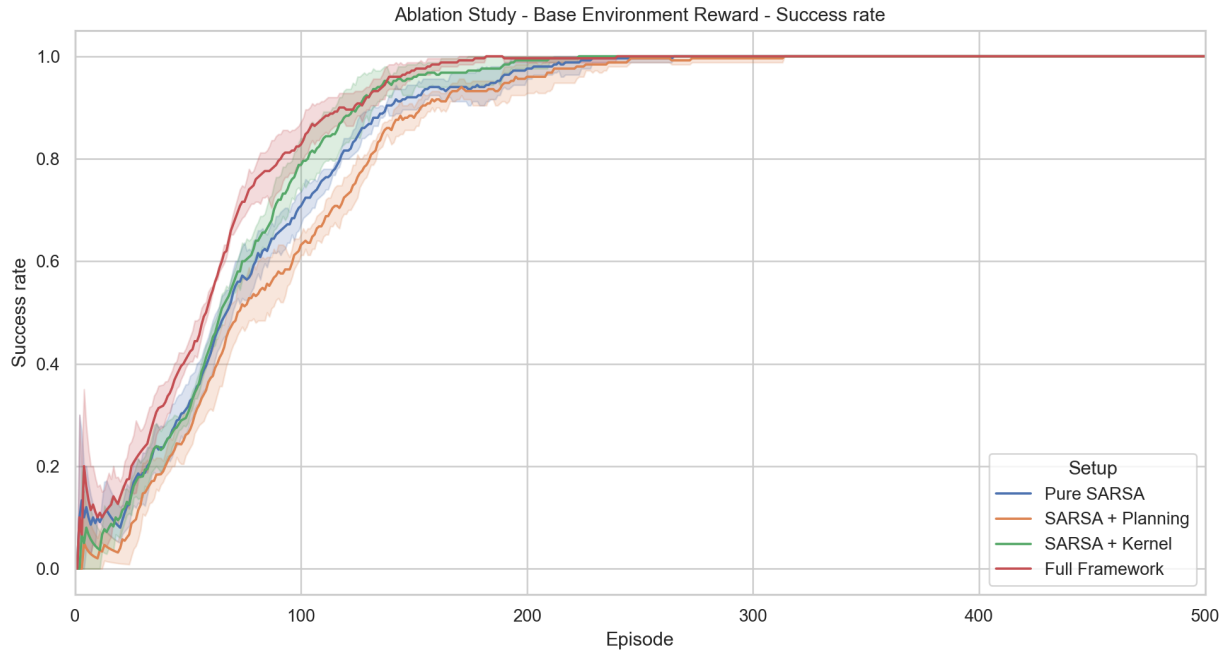

Figure: Ablation base success rate

In the final 50-episode tail, all four base-reward setups reached 100% success in the current results. The final mean environment reward was approximately:

```
Pure SARSA:          4.98
SARSA + Planning:    2.73
SARSA + Kernel:      6.66
Full Framework:      5.92
```

The kernel-only setup was strongest in final environment reward. This makes sense in deterministic Taxi because nearby states with the same passenger status and destination often share similar movement values. Planning alone was weaker in the final score, likely because replay can amplify early imperfect values and because the fixed planning depth may not be optimal for every phase of learning. The full framework still performs well, but its extra mechanisms increase sensitivity to hyperparameters.

9.2 Reward Comparison

The reward comparison uses the full framework for all reward designs. Since internal training rewards have different scales, the main environment reward plot is normalized.

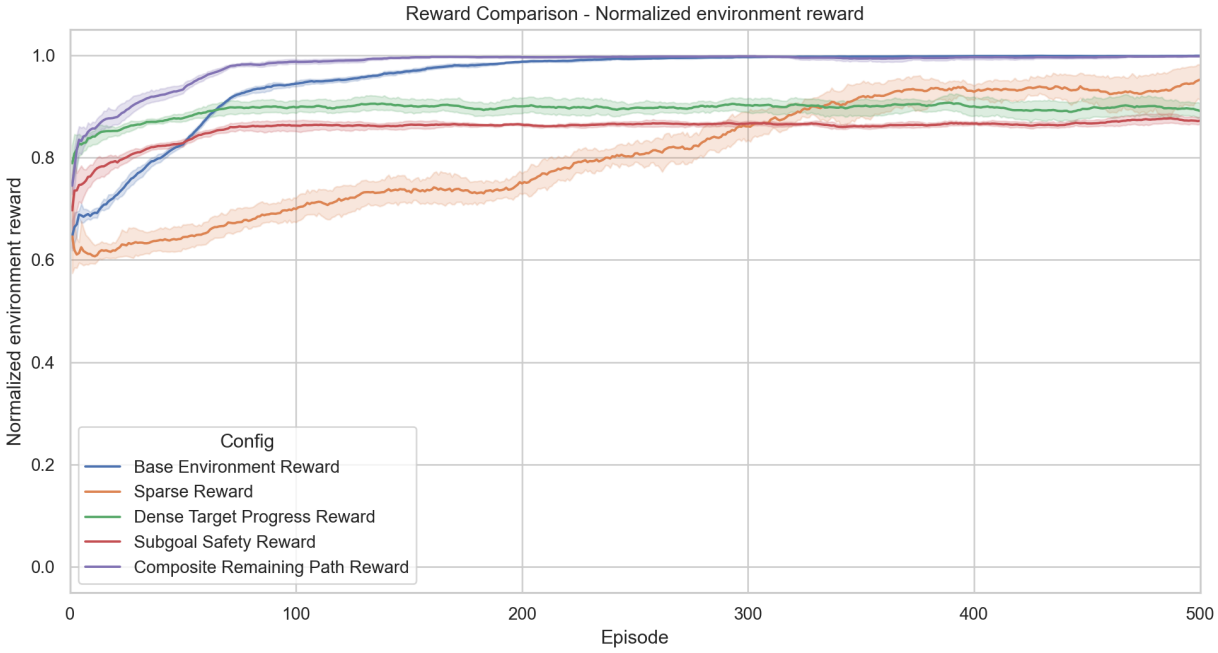

Figure: Reward comparison normalized environment reward

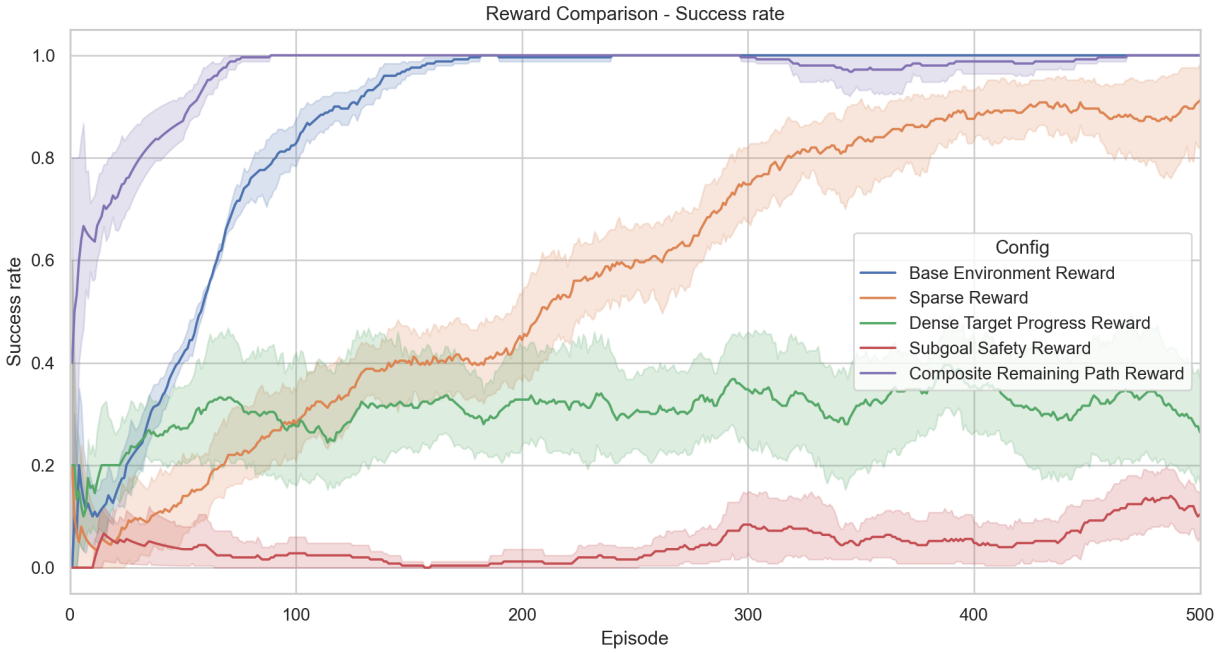

Figure: Reward comparison success rate

Over the final 50 episodes per seed, the approximate results were:

```
Base:       env 5.92,    success 1.000, illegal 0.08, normalized 0.999
Sparse:     env -87.62,  success 0.912, illegal 7.94, normalized 0.952
Reward 3:   env -211.02, success 0.264, illegal 7.12, normalized 0.891
Reward 4:   env -249.19, success 0.104, illegal 7.62, normalized 0.872
Reward 5:   env 3.82,    success 1.000, illegal 0.09, normalized 0.998
```

The important conclusion is not that custom rewards are always better. Reward 5 worked very well because it gave a global route-shortening signal while keeping the scale moderate. Sparse reward eventually learned in many episodes, but its weak signal left many illegal actions. Rewards 3 and 4 had reasonable design logic, but under the shared hyperparameters they were harder to optimize. Reward 4's strong penalties seem especially sensitive to the learning rate and planning depth, which is supported by the parameter sweep.

9.3 Parameter Sweep

The parameter sweep compares learning rate and planning depth for the base reward and Reward 4.

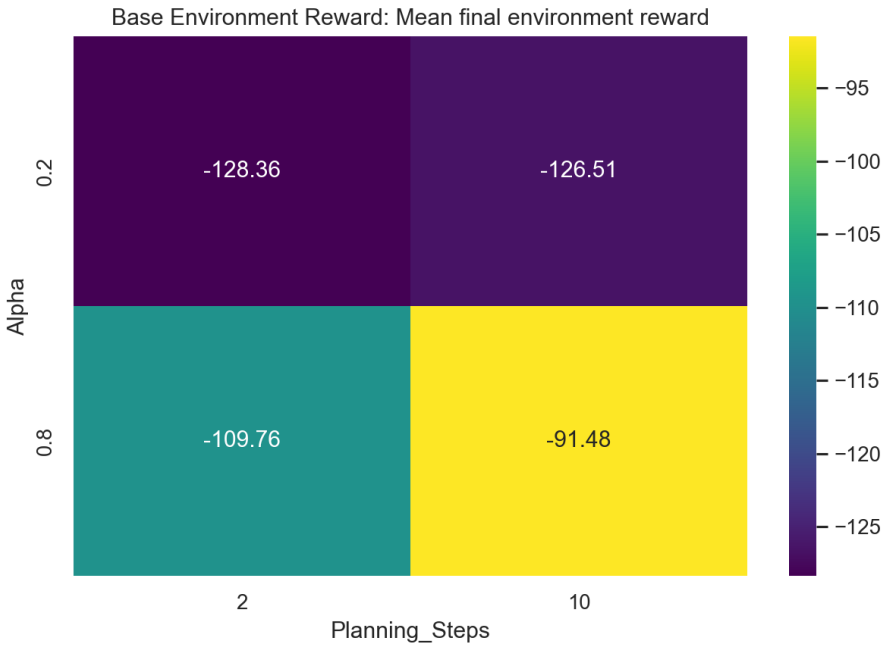

Figure: Parameter sweep base heatmap

Figure: Parameter sweep reward 4 heatmap

The strongest tested base setting was alpha = 0.8 with n = 10, with final mean environment reward around -91.48 and final success around 0.968. The strongest tested Reward 4 setting was alpha = 0.8 with n = 2, with final mean environment reward around -107.56 and final success around 0.912.

This result shows that the same hyperparameters are not equally good for every reward. Reward 4 has sharper penalties and therefore needs a learning rate that moves values enough to overcome early negative experience. However, increasing planning depth from 2 to 10 did not help Reward 4 in the tested settings,

suggesting that planning can replay penalty-heavy estimates too aggressively.

10. Surprising Results and Agent Behavior

One surprising behavior was that some saved policies entered loops even when other seeds succeeded. This happened during greedy visualization, where epsilon is set to zero. A learned Q-table can be strong on average but still contain local action preferences that cycle for a particular start state.

Another surprising result was that the most carefully penalized reward, Reward 4, did not dominate in the reward comparison. Its design is logical, but strong penalties can make the learning signal harsh. If early exploration produces many illegal actions, planning can replay those negative updates and slow down useful behavior unless the hyperparameters are tuned.

Finally, the Taxi animation appears to jump between cells. This is expected because Taxi-v4 is a discrete gridworld. The renderer displays one state per action; it does not simulate continuous driving.

11. Exploration vs. Exploitation

Exploration is most difficult for sparse reward. The agent receives almost no signal until the final dropoff, so early exploration often looks random and unproductive. Dense rewards reduce this burden by giving feedback before the full task is solved.

Base reward gives a useful balance because every step is penalized, illegal actions are strongly penalized, and successful dropoff gives a large positive reward. Reward 5 also gives useful intermediate feedback because reducing the remaining task distance is rewarded.

Prioritized sweeping changes the exploration-exploitation tradeoff indirectly. It lets the agent exploit its internal model by replaying important transitions, but those transitions come from past exploration. If early exploration discovers useful transitions, planning accelerates learning. If early exploration mostly discovers bad transitions, planning may replay those too.

Kernel generalization also changes the tradeoff. It exploits spatial similarity: learning from one taxi position can help nearby compatible positions. This improves sample efficiency, but only because the kernel is restricted to states with the same passenger status and destination.

12. Pragmatic Engineering Choices

Several practical choices were made to keep the project runnable:

- Kernel neighbors are precomputed once when the agent is created.
- Only the closest kernel neighbors are used.
- The priority queue is bounded.
- Kernel generalization is restricted to movement actions.

- Kernel updates are not applied inside every planning step by default.
- Plots use rolling mean smoothing to make noisy learning curves readable.
- Model artifacts are saved so visualization does not require retraining.
- A one-seed command and a short `--fast` command are provided for checking.

These choices reduce runtime while preserving the core scientific comparisons.

13. Quick Start

Install dependencies:

```
pip install -r requirements.txt
```

Run the full experiment:

```
python main.py
```

Run a one-seed check:

```
python main.py --seed 1
```

Run a short development run:

```
python main.py --fast
```

Visualize saved policies:

```
python main.py --visualize-saved
```

Explain one saved episode step by step:

```
python explain_simulation.py --load-saved --agent full --reward base --seed 1
```

Important files:

```
agent.py       - SARSA, prioritized sweeping, kernel generalization
main.py        - rewards, experiments, plots, model saving, CLI
visualize.py   - Gymnasium Taxi visualization
results/       - CSV files, plots, saved models
reports/       - report artifacts
```

14. Project Structure

```
RL_PROJECT/
  agent.py
  main.py
  visualize.py
  explain_simulation.py
```

```
requirements.txt
results/
  component_ablation.csv
  reward_comparison.csv
  parameter_sweep_seed_results.csv
  parameter_sweep_summary.csv
  *.png plots
  models/*.npz
reports/
  final_report.md
  final_report.pdf
  final_report.docx
```

15. Additional Insights

Reward design should be evaluated with external metrics, not only the internal reward used for training. Otherwise, a reward with a larger numerical scale can look better even when the learned policy is worse. This is why the report emphasizes original environment reward, normalized environment reward, success rate, and illegal actions.

The best learning setup depends on the reward. Planning and generalization are not universally beneficial in the same way for every reward function. They can improve sample efficiency, but they can also amplify biased or noisy estimates. In Taxi, the most reliable improvements came from combining a meaningful intermediate signal with careful restrictions on generalization.

16. Conclusion

This project shows that reward design can substantially change the behavior of a SARSA-based Taxi agent. Sparse reward is simple but weak. Dense progress rewards are intuitive but can be sensitive to scale and penalties. Subgoal safety rewards can reduce bad behavior, but they require careful tuning. The Composite Remaining Path reward performed well because it gave a global signal aligned with the full task while staying relatively stable.

Prioritized sweeping improves learning by focusing model updates on high-error transitions. Kernel generalization spreads value information across nearby states, but only works safely when the states are semantically compatible. The final system is reproducible, supports 5-seed evaluation with confidence intervals, saves learned policies, and provides commands for both full runs and quick single-seed checks.